

A MODEL THAT DECIDES INSTEAD OF WRITING



The full explainer

Jev takes the state your program already holds and a list of typed questions, and hands back the answers. No sentence around them, nothing to parse, nothing to retry. Most calls land near 100 ms.

01 MOST MODEL CALLS ARE DECISIONS, NOT ESSAYS

is this ticket billing? is this diff risky?

WHAT YOU WRITE TODAY

- 1 Write a prompt that asks for one word back.
- 2 The model writes a paragraph, or a JSON blob.
- 3 Parse it. Check the parse worked.
- 4 Handle the run where it did not.

WHAT JEV TAKES

- 1 The state: the ticket, the file, the last six turns.
- 2 The questions, each with its answers listed up front.
- 3 Back come the answers, typed, with probabilities.
- 4 The typed answer goes straight into an if statement.

The text in the middle is overhead, and you pay for it twice: in tokens, and in the seconds somebody spends waiting. Jev deletes the middle. It writes no prose at all, so it cannot replace the model that does.

70–500 ms

End to end. Most calls land near 100 ms.

\$0.042

Per million input tokens. Output tokens are free.

0%

Malformed answers, by construction rather than measurement.

12.2x

Cheaper than 13 separate calls on one 53,777-character document.

02 THREE QUESTION TYPES, ONE ROUND TRIP

nominal, ordinal, binary · mix them in one call

Choice NOMINAL

Pick one option from a set you write out. Up to 255 of them.

.choice · .probabilities · .confidence

Score ORDINAL

Place the state on a rubric you order yourself. 1.4 is a real answer.

.score · .probabilities · .confidence

Noul BINARY

Is this statement true of the state? Answered as a probability.

.noul — the number is the confidence

03 ONE TICKET, THREE ANSWERS, THREE THRESHOLDS

illustrative values, not a recorded run

```
# "I was charged twice for order A-104. Please refund the duplicate today."
answers = client.system_one(
    model="jev-1.13.0", state={"ticket": ticket},
    questions={
        "team": Choice(criteria={"billing":.., "technical":.., "sales":..}),
        "anger": Score(criteria=["calm", "frustrated", "furious"]),
        "wants_refund": Noul(instructions="asking for money back"),
    }).answers
# team "billing" 0.94 | anger 1.4 / 2 | wants_refund 0.97
```

< 0.60

Queue it for a human. The model is telling you it does not know.

> 1.20

Angry enough to jump the queue. Mark it priority.

> 0.90

Open the refund case. Money moves, so the bar is high.

04 HOW IT GETS THERE, AND WHAT EACH CHOICE COSTS

three design decisions

Non-autoregressive

All outputs come from a single pass instead of one token feeding the next, so twelve questions cost the tokens of twelve and the wall-clock of one.

Cost: questions cannot build on each other. If B needs A's answer, that is two calls.

Fixed output space

You list the options before the call, so the model chooses among your strings. A malformed answer has nowhere to live.

Cost: a confidently wrong answer arrives in the same well-formed shape as a right one.

Trained for calibration

Reinforcement learning aimed at probabilities that match outcomes, not at a helpful-sounding reply.

Cost: every gate you build inherits that claim, so test it on your own data.

05 RUN IT AGAINST YOUR OWN FILES IN TWENTY MINUTES

one dependency, one key, one file

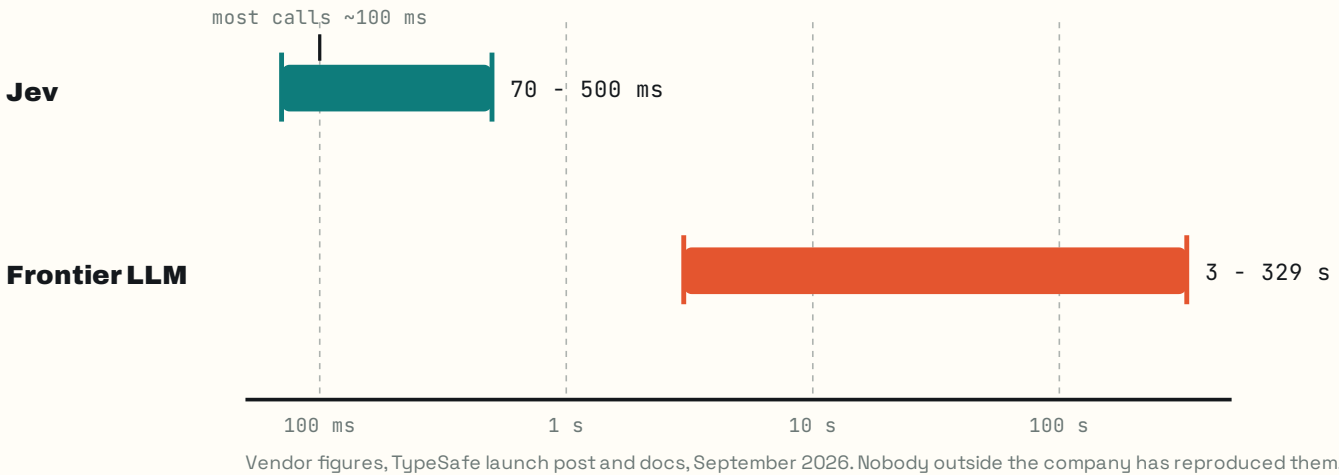
```
pip install typesafe-sdk && export TYPESAFE_API_KEY="sk-..."
python readme_check.py https://github.com/psf/requests
# written for user (0.94)
# setup steps 1.4 / 2 worked example 0.22
```

Point it at twenty READMEs you already know. You are not testing whether Jev is clever. You are testing the one claim everything rests on: that when it says 0.9 it is right about nine times in ten, on your material.

06 THE GAPI IS THE WHOLE ARGUMENT

log scale · vendor figures

TIME TO ONE DECISION · LOG SCALE



07 WHAT IT GETS WRONG

TypeSafe publishes these; most model cards do not

- ✗ **It cannot count.** Characters, occurrences, items in a list.
- ✗ **It reads literally.** "This is not a refund request" scores high on a refund question.
- ✗ **User text steers it.** Treat state built from user input the way you treat a prompt.
- ✗ **Date and number comparisons fail.** Do that part in Python.
- ✗ **Padding costs accuracy.** State the question did not need drags the answer down.
- ✗ **No text, ever.** No explanations, no code, no images, audio or video.

Accuracy is 67.8% on TypeSafe's own benchmark, scored against consensus labels from GPT-6 and Claude. TypeSafe ran that evaluation itself, so reproduce it on your data before you put a threshold in production.

08 PATTERNS THAT HOLD UP

the shape of the code around the call

FAN OUT. Ask everything you might want in one request. The state is what costs, and it is sent once.

GATE PER ACTION. One threshold for a whole system is wrong nearly everywhere. Showing a balance can run at 0.5; moving money cannot.

COMPOSITE SCORING. Split a judgment into dimensions and weight them in your repo, where a diff can show the weights.

CASCADE. Jev routes, plain code handles the easy cases, a frontier model takes the rest and sees a fraction of the traffic.

09 WORTH AN HOUR EACH

a few hundred projects; most are wrappers

<code>winnow</code>	Splits tool output into 25-line blocks and asks Jev which ones the agent needs. Context pruning done with judgment.
<code>foreman</code>	Nine noulis per cycle watching a coding agent. Supervision as a second loop that never touches tool choice.
<code>openjev-sglang</code>	The same API on open weights, one-token logprob readout. The clearest account of how a call like this works.
<code>jevcal</code>	Fits confidence thresholds to your own data, and guards them in CI against drift.

10 THE WHOLE API

POST /v1/systemone

WHAT YOU SEND

`model` Pin it. `jev-1.13.0`, not `jev-latest`, once a threshold is tuned.

`state` A string, an object or an array of text. Use an object: named fields tell the model which part is which. No images.

`limits` 64k tokens for the whole request; 32k for the state plus any one question. The state is the budget.

`questions` A map of your own ids to Choice, Score and Noul.

WHAT COMES BACK

`answers` Keyed by the ids you chose. `.choice` `.score` `.noul`

`probs` The full distribution. Log it: a run of 0.51 answers that all came out right tells you something stored labels cannot.

`usage` Input tokens, and output tokens at zero.

No session, no thread, no conversation history to manage, because nothing carries over from one call to the next. Every call is a function call with its arguments in front of it.

Demonstration and learning only. Every figure here is a vendor number published by TypeSafe in September 2026, and nobody outside the company has reproduced the latency. The example values are drawn to make a point, not measured. Pin the model version before you tune a threshold, and run your own evaluation before any of this reaches production.



READ THE FULL EXPLAINER

github.com/aarora79/my-ai-assets-explainers/jev/

Sources: TypeSafe launch post and documentation; `openjev-sglang`; LangChain on building a harness with Jev; `cobanov/awesome-jev`. Every number on this poster is dated and attributed in the explainer.